# Uso de JavaScript (I): Manipulación del DOMy renderizadores


## Universidad de Sevilla

### Departamento de Lenguajes y Sistemas Informáticos

Carlos Arévalo, Daniel Ayala, José Calderón, Margarita Cruz, Inma Hernández, David Ruiz


UNIVERSIDAD DE SEVILLA

# Contenidos

Modelo DOM

Manipulación de DOM mediante JavaScript

Renderizadores

# Contenidos

**Modelo DOM**

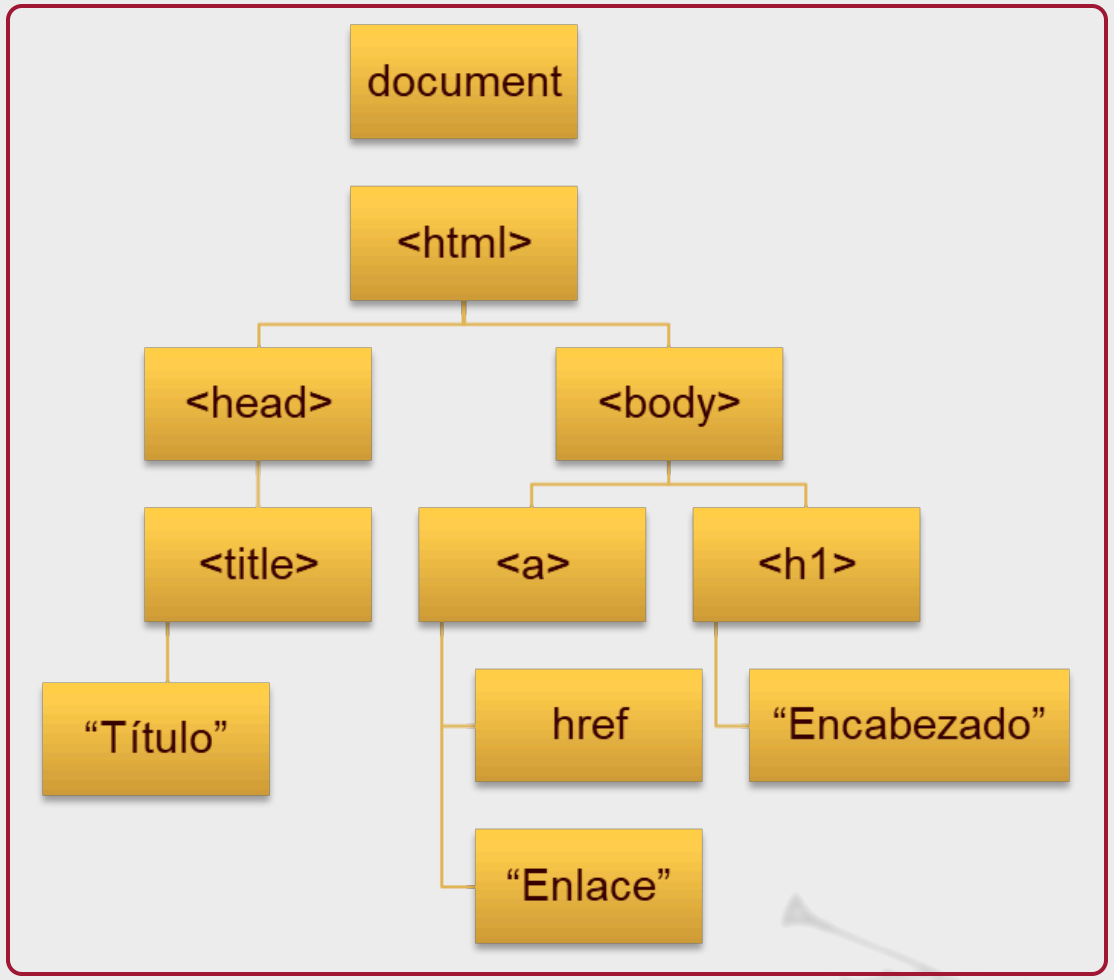
Manipulación de DOM mediante JavaScript

Renderizadores

# Modelo DOM. Document Object Model

Cuando una página web se carga en el navegador, se crea un modelo de dicha página en forma de árbol (árbol DOM)

El árbol DOM contiene una jerarquía de objetos, que representan los distintos elementos contenidos en la página



# Definición (W3C)

> "The Document Object Model is a platform- and language-neutral interface that will allow programs and scripts to dynamically access and update the content, structure and style of documents."

API

- Cross-platform
- Language-independent

Permite dinámicamente acceder y modificar

- Contenido
- Estructura
- Estilos

# Manipulación del DOM con JavaScript

A partir del modelo DOM de la página, JavaScript puede modificarla de forma programática en tiempo de ejecución, dando lugar a HTML dinámico.

Concretamente JavaScript puede

- Cambiar los elementos HTML de la página
- Cambiar los valores de los atributos de dichos elementos
- Cambiar los estilos CSS asociados a los elementos
- Eliminar elementos y atributos existentes en el HTML
- Añadir nuevos elementos y atributos HTML
- Ejecutar funciones y scripts en respuesta a los eventos originados en los elementos HTML
- Crear nuevos manejadores de eventos
- Eliminar manejadores de eventos
- …

# Modelo DOM

Recomendación W3C: https://www.w3.org/DOM/DOMTR

Interfaz de programación

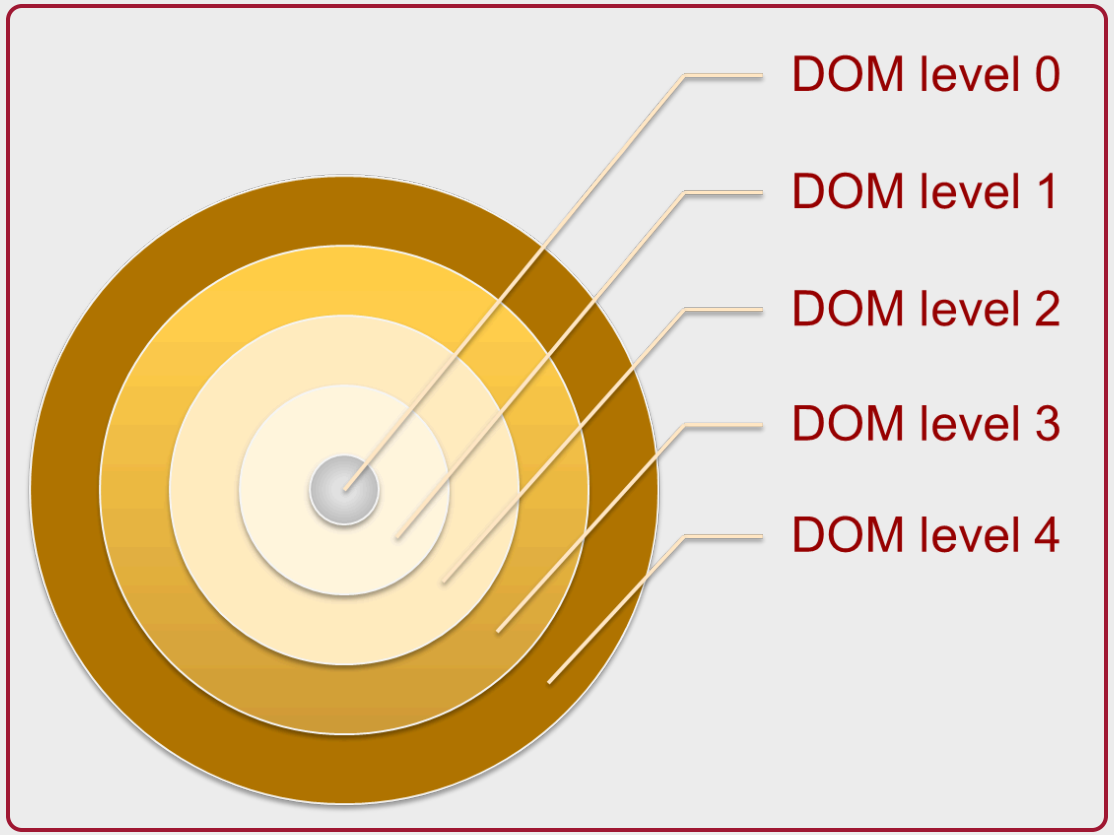
- Los elementos HTML como objetos
- Las propiedades de todos los elementos HTML
- Los métodos para acceder a todos los elementos HTML
- Los eventos para todos los elementos HTML

Dividido en tres partes

- Core DOM – Modelo estándar para todos los tipos de documentos
- XML DOM - Modelo estándar para documentos XML
- HTML DOM - Modelo estándar para documentos HTML

# Niveles DOM

- Después del Level 4, el estándar pasó a ser un WhatWG Living standard
- Cada nivel incluye el nivel anterior y lo incrementa
- El nivel 0 no fue recomendación, se usó de manera informal



# Niveles DOM

DOM level 2

- CORE
- Views
- Events
- Style
- Traversal and range
- HTML

DOM level 3

- Validation
- Load and Save

# DOM Level 0

Aparece cuando empieza a haber soporte para JavaScript en los navegadores (Netscape)

Permite acceder a **algunos elementos** HTML, principalmente formularios e imágenes

Es común a todos los **navegadores**: se soporta de la misma forma en todos ellos

- En niveles posteriores, esto no es así

La mayoría de navegadores sigue manteniendo el soporte de este nivel, pese a **que no sea una recomendación W3C**

# DOM Level 1

Publicado en 1998

Definido en principio para documentos XML, aunque también es applicable a HTML

Permite acceder al documento completo

Define un conjunto de colecciones de objetos

- Images, applets, links, forms, anchors, …

Documentación técnica

- Core: https://www.w3.org/TR/REC-DOM-Level-1/level-one-core.html
- HTML: https://www.w3.org/TR/REC-DOM-Level-1/level-one-html.html

# DOM Level 2

Publicado en 2001

Documentación técnica

- Core: https://www.w3.org/TR/2000/REC-DOM-Level-2-Core-20001113/
- Views: (ya obsoleto, precursor del concepto actual de “presentación”) https://www.w3.org/TR/2000/REC-DOM-Level-2-Views-20001113/
- Events: (tipos de eventos, capturing/bubbling, registro de manejadores y listeners, …) https://www.w3.org/TR/2000/REC-DOM-Level-2-Events-20001113/
- Style: (hojas de estilo, CSS) https://www.w3.org/TR/2000/REC-DOM-Level-2-Style-20001113/
- Traversal and Range: (Recorrido de los nodos del árbol) https://www.w3.org/TR/2000/REC-DOM-Level-2-Traversal-Range-20001113/
- HTML: https://www.w3.org/TR/2003/REC-DOM-Level-2-HTML-20030109/html.html

# DOM Level 3

Publicado en 2004

Introduce

- XPath
- Manejo de eventos de teclado
- Validación del documento con relación a un esquema DTD de referencia (método validateDocument)

Documentación técnica

- Core: https://www.w3.org/TR/2004/REC-DOM-Level-3-Core-20040407/
- Load & Save: (serialización de objetos) https://www.w3.org/TR/2004/REC-DOM-Level-3-LS-20040407/
- Validation: https://www.w3.org/TR/2004/REC-DOM-Level-3-Val-20040127/

# DOM Level 4

Publicado en 2015

Es un snapshot del living standard

Centrado en el concepto de MutationObserver, un objeto que observa y reacciona ante mutaciones de nodos del árbol

Documentación técnica

- Core: https://www.w3.org/TR/domcore/

# DOM Living standard

Última actualización: 15 de enero de 2026

Documentación técnica

- https://dom.spec.whatwg.org/

# Compatibilidad de motores JavaScript.

El soporte a los distintos métodos y atributos varia según el navegador

| API / Propiedad | IE | Firefox | Safari | Chrome | Opera |
|---|---|---|---|---|---|
| `innerHTML` | IE10 | Sí | Sí | Sí | Sí |
| `innerText` | Sí | No | Sí | Sí | Sí |
| `classList` | IE10 | Sí | Sí | Sí | Sí |
| `moveRow` | Sí | No | No | No | No |
| `Id` | Sí | Sí | Sí | Sí | Sí |
| `activeElement` | IE 8 | 32Win, 32Linux | Incompleta | Incompleta | Incompleta |
| `parentWindow` | Sí | No | No | No | No |
| `doctype` | No | Sí | Sí | Sí | Sí |

(Fuente: https://quirksmode.org/dom/html/)

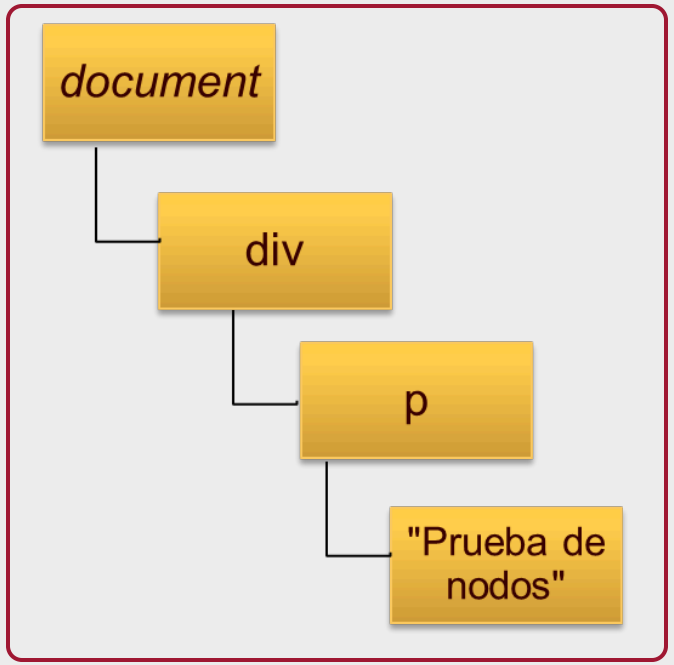
# Node

En DOM, todo objeto es un nodo (node)

- Incluye elementos, textos y comentarios

Ejemplo:

```
<div>
  <p>
    Prueba de nodos
  </p>
</div>
```




Cuatro nodos: `document`, `<div>`, `<p>`, Texto dentro de `<p>`

`document` es siempre el nodo raíz del DOM

El nodo `<p>` es hijo (child) de `<div>` y padre (parent) del texto

# Jerarquía de nodos

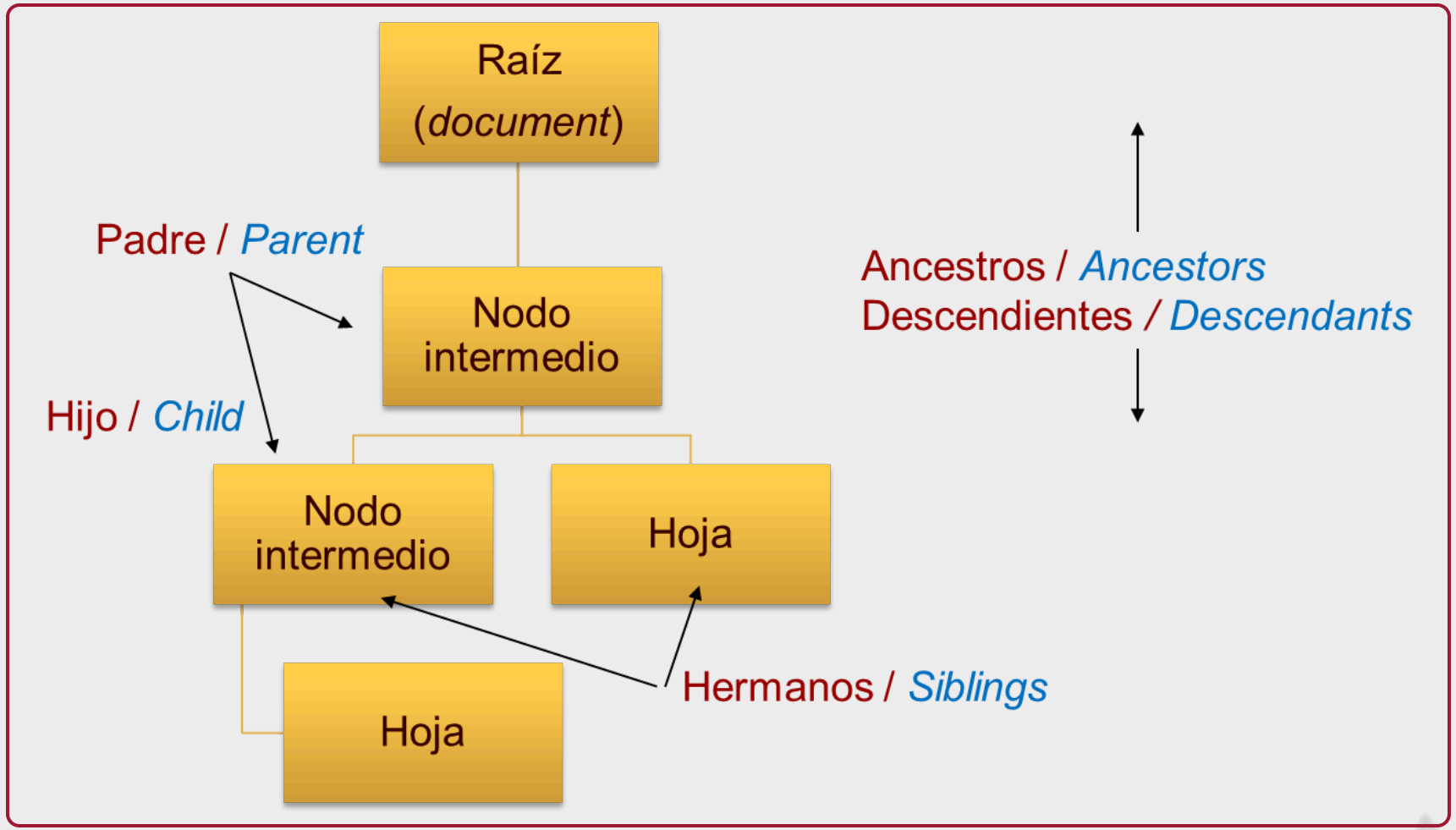
Cada nodo tiene exactamente un padre, excepto el raíz (que no tiene padre)

Cada nodo puede tener un número de hijos (ninguno, uno o varios)

- Cuando el nodo no tiene hijos se denomina hoja (leaf)
- Cuando el nodo tiene hijos y padre se denomina nodo intermedio

Si dos nodos tienen el mismo padre se consideran hermanos

# Jerarquía de nodos

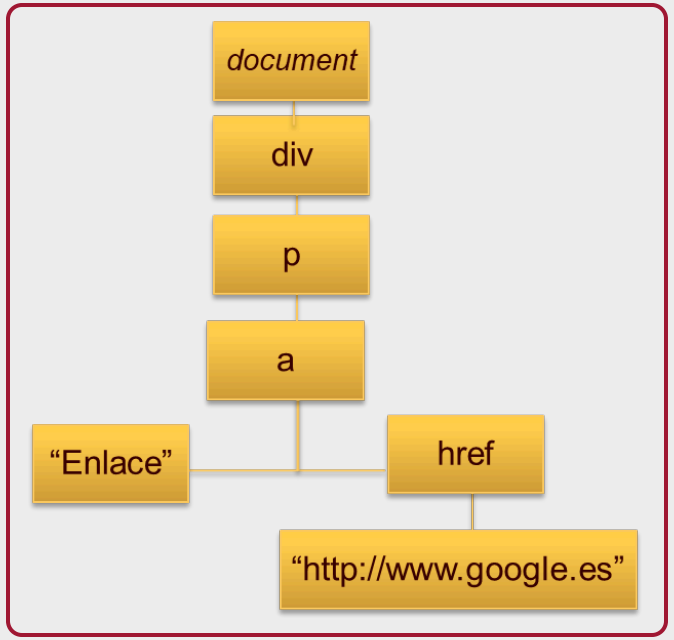


# Attribute node

Los atributos de los elementos también son nodos, hijos del nodo elemento.

Ejemplo:

```
<div>
  <p>
    <a href="http://www.google.es">
      Enlace
    </a>
  </p>
</div>
```



# Grafo - tipos de nodos

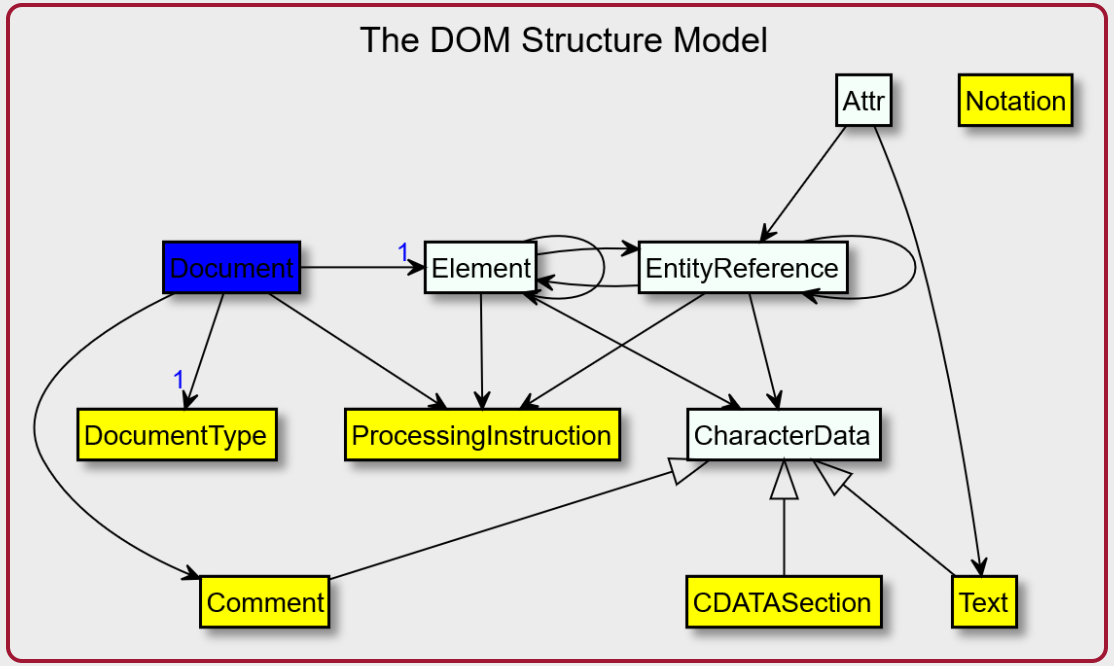
### 1.1.1 The DOM Structure Model

The DOM presents documents as a hierarchy of Node objects that also implement other, more specialized interfaces. Some types of nodes may have child nodes of various types, and others are leaf nodes that cannot have anything below them in the document structure. For XML and HTML, the node types, and which node types they may have as children, are as follows:

- Document -- Element (maximum of one), ProcessingInstruction, Comment, DocumentType (maximum of one)
- DocumentFragment -- Element, ProcessingInstruction, Comment, Text, CDATASection, EntityReference
- DocumentType -- no children
- EntityReference -- Element, ProcessingInstruction, Comment, Text, CDATASection, EntityReference
- Element -- Element, Text, Comment, ProcessingInstruction, CDATASection, EntityReference
- Attr -- Text, EntityReference
- ProcessingInstruction -- no children
- Comment -- no children
- Text -- no children
- CDATASection -- no children
- Entity -- Element, ProcessingInstruction, Comment, Text, CDATASection, EntityReference
- Notation -- no children

The DOM also specifies a NodeList interface to handle ordered lists of Nodes, such as the children of a Node, or the elements returned by the Element.getElementsByTagNameNS(namespaceURI, localName) method, and also a NamedNodeMap interface to handle unordered sets of nodes referenced by their name attribute, such as the attributes of an Element. NodeList and NamedNodeMap objects in the DOM are *live*; that is, changes to the underlying document structure are reflected in all relevant NodeList and NamedNodeMap objects. For example, if a DOM user gets a NodeList object containing the children of an Element, then subsequently adds more children to that element (or removes children, or modifies them), those changes are automatically reflected in the NodeList, without further action on the user's part. Likewise, changes to a Node in the tree are reflected in all references to that Node in NodeList and NamedNodeMap objects.

Finally, the interfaces Text, Comment, and CDATASection all inherit from the CharacterData interface.



# Contenidos

Modelo DOM

**Manipulación de DOM mediante JavaScript**

Renderizadores

# Manipulación de DOM mediante JavaScript

DOM ofrece una API con métodos y propiedades para el acceso y la manipulación de los elementos y atributos HTML

El primer paso para manipular un elemento/atributo HTML es encontrar el elemento (obtener una referencia al objeto)

A continuación se pueden aplicar algunos métodos para modificar, añadir y eliminar elementos y atributos

El objeto document es la raíz de todos los objetos de la página

- Para acceder a cualquier elemento de la página, el punto de partida es document

# Métodos DOM – Encontrar elemento

`node.getElementById(id)` : Obtiene un elemento descendiente del nodo a partir de su `ID`

- Ejemplo: `document.getElementById("idElemento")`

`node.getElementsByTagName(tagName)` : Obtiene un array de descendientes del nodo que sean un tipo de etiqueta

- Ejemplo: `document.getElementsByTagName("a")`

`node.getElementsByClassName(className)` : Obtiene un array de descendientes del nodo con una clase determinada

- Ejemplo: `document.getElementsByClassName("nombreClase")`

`node.querySelectorAll(cssSelector)` : Obtiene un array de descendientes del nodo que concuerdan con un determinado selector `CSS`

- Ejemplo: `document.querySelectorAll("div > p.clase")`

# Métodos DOM – Consultar elementos y atributos

`node.classList` : Devuelve la lista de las clases del nodo, provee métodos para añadir, eliminar y alternar clases

`node.tagName` : Devuelve el tipo de etiqueta `HTML` del nodo

`node.style` : Proporciona acceso para consultar y modificar los estilos `CSS` del nodo mediante atributos

`node.innerHTML` : Devuelve y permite modificar el contenido `HTML` de un nodo

`node.innerText` : Devuelve y permite modificar el contenido textual de un nodo

`node.getAttribute("atributo")` : Permite consultar el valor de un atributo del nodo

# Métodos DOM – Modificar elementos y atributos

`node.setAttribute(nombreAtributo, valor)`: Modifica el valor de un atributo

`document.createElement(nombreElemento)`: Crea un nuevo elemento `HTML` (sólo funciona sobre `document`)

`node.removeChild(elemento)`: Elimina un nodo hijo

`node.remove()`: Elimina un nodo del lugar donde esté

`node.append(elemento)`: Añade un elemento como último nodo hijo

`node.prepend(elemento)`: Añade un elemento como primer nodo hijo

`node.insertBefore(elemento, hermano)`: Añade un elemento como nodo hijo, por delante de un nodo hermano

`node.replaceChild(nuevo, antiguo)`: Reemplaza un nodo hijo con otro

# Métodos DOM – Navegar por la jerarquía

`node.parentNode` : Cuando se invoca sobre un objeto, devuelve el nodo padre

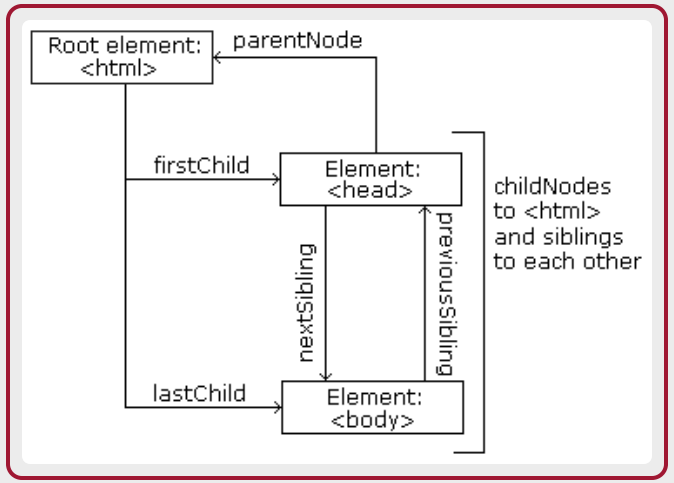
`node.firstChild` : Devuelve el primer hijo de un nodo

`node.lastChild` : Devuelve el último hijo de un nodo

`node.childNodes[i]` : Devuelve el `i` -ésimo hijo de un nodo

`node.nextSibling` : Devuelve el siguiente hermano de un nodo

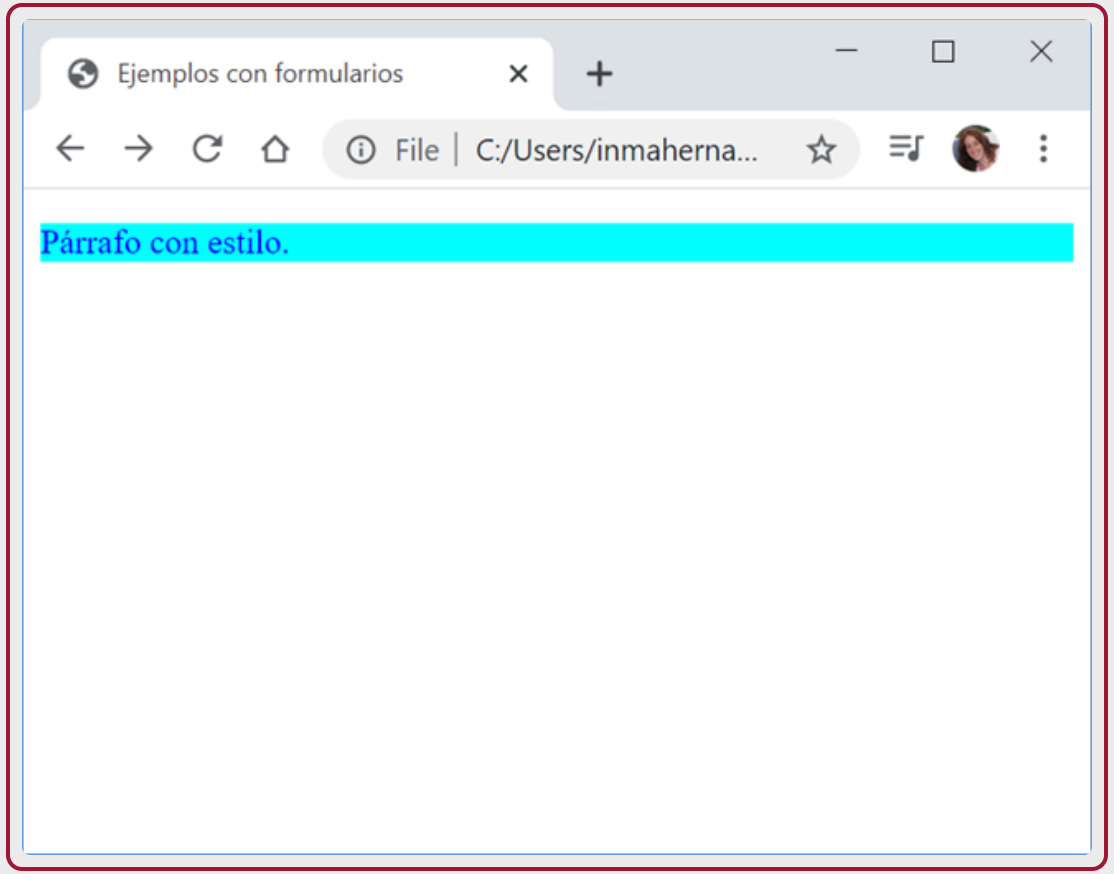
`node.previousSibling` : Devuelve el hermano anterior de un nodo



# Ejemplo – Cambio de estilo

```
<body>
  <p id="par">Párrafo con estilo.</p>
</body>
```

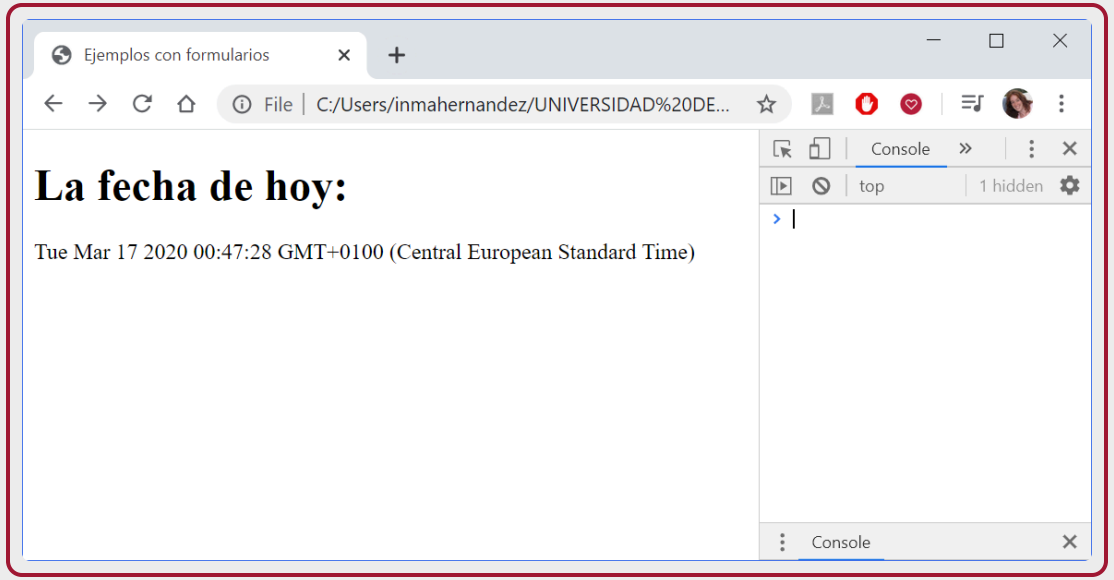
```
let paragraph = document.getElementById("par");
paragraph.style.color = "blue";
paragraph.style.background = "cyan";
```



# Ejemplo – Escribir la fecha actual

```
<body>
  <h1>La fecha de hoy:</h1>
  <span id="date"></span>
</body>
```

```
let dateSpan = document.getElementById("date");
let date = new Date();
dateSpan.innerText = date.toString();
```



# Ejemplo – Crear una lista dinámicamente

```html
<body>
  <ul id="lista"></ul>
</body>
```

```javascript
let lista = document.getElementById("lista");
let n_items = 4;

for (let i = 0; i < n_items; i++) {
  let listItem = document.createElement("li");
  listItem.innerText = "Elemento " + i;
  lista.appendChild(listItem);
}
```

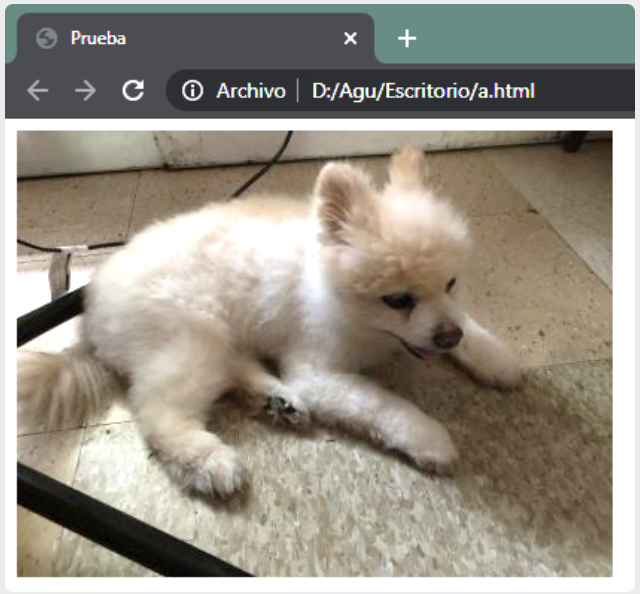
- Elemento 0
- Elemento 1
- Elemento 2
- Elemento 3

# Ejemplo – Crear un elemento HTML desde cadena

```
function parseHTML(str) {
  // Creamos un HTML "auxiliar"
  let aux = document.implementation.createHTMLDocument();
  // Insertamos la cadena HTML en el documento
  aux.body.innerHTML = str;
  // Devolvemos el elemento raíz
  return aux.body.children[0];
}
```

```
<body>
  <div id="image-container"></div>
</body>
```

```
let html = '<img src="images/perrito.jpg">';
let img = parseHTML(html);
let container = document.getElementById("image-container");
container.appendChild(img);
```



# Otras propiedades DOM (I)

`document.URL`

`document.body`

`document.cookie`

`document.doctype`

`document.head`

`document.inputEncoding`

`document.readyState`

`document.title`

(…)

# Otras propiedades DOM (II) - NodeLists

Al cargarse una página en el navegador, se crean varias propiedades para acceder a las colecciones de objetos:

- `document.images`: todas las imágenes del documento.
- `document.forms`: Todos los formularios del documento.
- `document.forms[X].elements`: Todos los campos del formulario `X`, sean del tipo que sea.
- `document.links`: Todos los enlaces de la página (`<a href>`).
- `document.scripts`: Todos los `scripts`

Las propiedades anteriores son de tipo HTMLCollection

- Muy parecido a un array (ej: método length, acceso mediante [])
- No se pueden usar los métodos valueOf(), pop(), push(), o join()

El acceso a cada elemento de la colección se puede hacer por número (la posición) o por nombre (el atributo “name” del elemento correspondiente).

# Contenidos

Modelo DOM

Manipulación de DOM mediante JavaScript

**Renderizadores**

# Renderizadores JS

Normalmente necesitamos mostrar muchas veces elementos de un mismo tipo, por ejemplo, fotos de una galería o mensajes de éxito o error.

O mostrar el mismo elemento de más de una forma diferente: por ejemplo, fotos en formato miniatura o con todos los detalles.

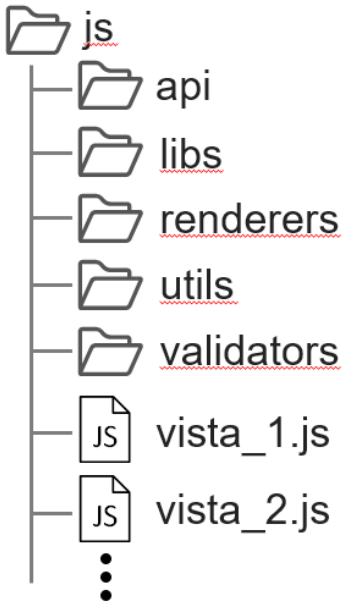
Un renderizador es un módulo JS que se especializa en proveer diferentes representaciones HTML de un tipo de elemento, dados sus atributos básicos

Se utiliza una cadena "plantilla" representando el HTML del elemento en la cual se insertan sus atributos y se convierte a un nodo DOM que se puede insertar en la página.

# Estructura del proyecto

## Organización del código

- Nosotros seguiremos la siguiente estructura




UNIVERSIDAD DE SEVILLA

# Ejemplo: renderizador de fotos

## Foto en formato miniatura:

```html
<div class="image-mini">
  <p class="image-title">Título de la imagen</p>
  <img src="images/1.png">
  <p class="image-desc">Descripción de la imagen</p>
</div>
```

```css
div.image-mini {
  border: 1px solid black;
  max-width: 200px;
}

div.image-mini > .image-title {
  font-size: 200%;
  font-weight: bold;
  text-align: center;
}

div.image-mini > img {
  max-width: 100%;
  height: auto;
}
```

# Ejemplo: renderizador de fotos

## Foto en formato completo:

```html
<div class="image-full">
  <h1 class="image-title">Titulo de la imagen</h1>
  <p class="image-desc">Descripción de la imagen</p>
  <img src="images/1.png">
</div>
```

```css
div.image-full {
  text-align: center;
}

div.image-full > .image-title {
  font-weight: bold;
  text-decoration: underline;
  font-family: 'Trebuchet MS';
}

div.image-full > .image-desc {
  font-style: italic;
  font-size: 120%;
}
```

# Ejemplo: renderizador de fotos

Reutilizando los formatos con un renderizador:

```
const photoRenderer = {
  asThumbnail: function(photo) {
    let html = `<div class="image-mini">
<p class="image-title">${photo.title}</p>
<img src="${photo.url}">
<p class="image-desc">${photo.description}</p>
</div>`;

    let elem = parseHTML(html);
    return elem;
  },

  asFull: function(photo) {
    let html = `<div class="image-full">
<h1 class="image-title">${photo.title}</h1>
<p class="image-desc">${photo.description}</p>
<img src="${photo.url}">
</div>`;

    let elem = parseHTML(html);
    return elem;
  }
};
```

# Ejemplo: renderizador de fotos

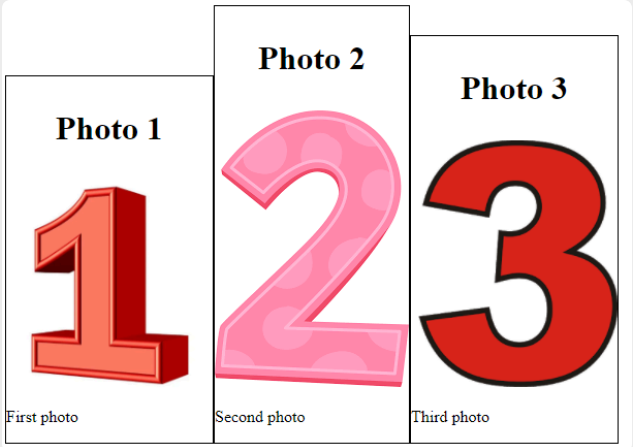
## Usando el renderizador para mostrar las fotos:

```
<body>
  <div id="photo-container"></div>
</body>
```

```
let photos = [
  { title: "Photo 1", description: "First photo", url: "images/1.png" },
  { title: "Photo 2", description: "Second photo", url: "images/2.png" },
  { title: "Photo 3", description: "Third photo", url: "images/3.png" },
];

let container = document.getElementById("photo-container");

for (let photo of photos) {
  let miniphoto = photoRenderer.asThumbnail(photo);
  container.appendChild(miniphoto);
}
```



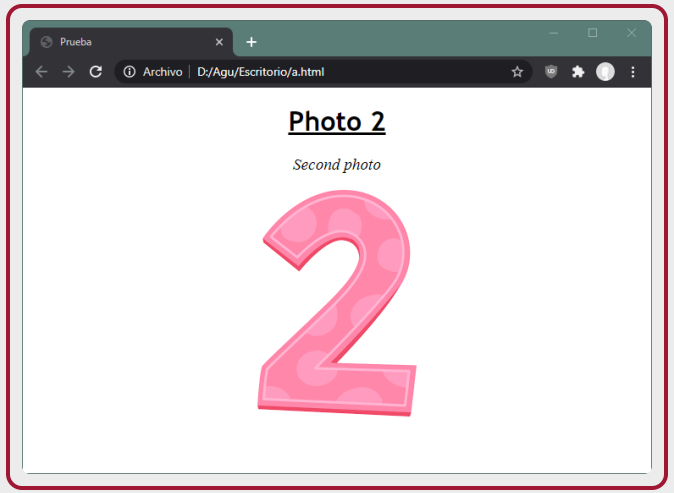
# Ejemplo: renderizador de fotos

## Usando el renderizador para mostrar las fotos:

```
<body>
  <div id="photo-container"></div>
</body>
```

```
let photos = [
  { title: "Photo 1", description: "First photo", url: "images/1.png" },
  { title: "Photo 2", description: "Second photo", url: "images/2.png" },
  { title: "Photo 3", description: "Third photo", url: "images/3.png" },
];

let container = document.getElementById("photo-container");
let bigPhoto = photoRenderer.asFull(photos[1]);
container.appendChild(bigPhoto);
```



# Ventajas y desventajas

Ventajas del uso de renderizadores

- Permiten separar datos de presentación/estilo
- Los datos pueden venir de cualquier parte: introducidos a mano, consulta a API, desde un archivo…
- Permiten representar la misma entidad de maneras diferentes
- Altamente reutilizables: se pueden usar en cualquier lugar de la aplicación
- Se puede encapsular en ellos código más complejo para dar formato a elementos o consultar datos adicionales para rellenar el elemento `HTML`
- Permiten centralizar los cambios si se desea modificar cómo se representa una entidad a lo largo de toda la aplicación

Desventajas

- Se debe encontrar primero la representación deseada escribiéndola en `HTML` "normal", y luego generalizarla
- Puede no merecer la pena para elementos que no aparezcan a menudo

# Conclusiones

El **DOM** representa el documento HTML como una **jerarquía de nodos** sobre la que JavaScript puede consultar y actuar.

JavaScript permite modificar de forma dinámica el **contenido**, la **estructura**, los **atributos**, los **estilos** y los **eventos** de una página web.

Las operaciones más habituales sobre el DOM incluyen **buscar nodos**, **recorrer relaciones de parentesco**, **crear elementos**, **insertarlos** y **eliminarlos**.

Trabajar con el DOM implica distinguir entre tipos de nodos y comprender cómo se organiza el árbol del documento.

Los **renderizadores** ayudan a separar **datos** y **presentación**, haciendo el código más **reutilizable**, **mantenible** y adaptable a distintos formatos de visualización.

Este enfoque resulta especialmente útil cuando una misma entidad debe mostrarse de formas distintas o cuando la interfaz se construye dinámicamente a partir de datos.

# Gracias

## Universidad de Sevilla

Departamento de Lenguajes y Sistemas Informáticos

UNIVERSIDAD DE SEVILLA